```
# asteroid.py
import os
os.environ["KERAS_BACKEND"] = "torch"
os.environ["KERAS_TORCH_DEVICE"] = "cpu"

import csv
import json
import uuid
import glob
import hashlib
from datetime import datetime
import yaml
import joblib
import numpy as np
import pandas as pd
from typing import Dict, Union, List, Any
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error
import keras
from keras import layers

def load_config(config_path="config.yaml"):
    if not os.path.exists(config_path):
        raise FileNotFoundError(f"Configuration file missing at: '{config_path}'")
    with open(config_path, "r") as f:
        return yaml.safe_load(f)

def _engineer_tabular_interactions(df: pd.DataFrame) -> pd.DataFrame:
    df_out = df.copy()
    if "H" in df_out.columns:
        df_out["H_squared"] = df_out["H"] ** 2
        df_out["brightness_log_scale"] = 10 ** (-0.2 * df_out["H"])
    if all(col in df_out.columns for col in ["a", "e", "i"]):
        df_out["orbit_log_volume"] = np.log1p(df_out["a"] * df_out["e"] * df_out["i"])
    return df_out

def _calculate_production_epochs(optimal_epochs: list, max_epochs_ceiling: int) -> int:
    if not optimal_epochs:
        return max_epochs_ceiling
    calculated_avg = int(np.ceil(np.mean(optimal_epochs) * 1.10))
    return min(calculated_avg, max_epochs_ceiling)

def _generate_metadata_payload(
    fitted_pipeline: Pipeline,
```

```
    feature_names: List[str],
    description: str,
    cv_metrics: dict,
    model_uuid: str,
    engine_type: str,
    target_col: str,
    hyperparameters: dict = None,
    mean_delta_cv_r2: float = 0.0,
    delta_threshold: float = 0.05,
    statistical_outcome: str = "H0_stat",
    operational_outcome: str = "H0_ops"
) -> dict:

    underlying_regressor = fitted_pipeline.named_steps["regressor"]

    # Handle Feature Importances Safely
    if hasattr(underlying_regressor, "feature_importances_"):
        importances = underlying_regressor.feature_importances_.tolist()
        imp_dict = dict(zip(feature_names, importances))
    else:
        imp_dict = {"info": "Permutation importance required for feature weights in neural
network configurations"}

    # Handle Hyperparameters Safely (Skl vs Keras)
    if hyperparameters is not None:
        params_payload = hyperparameters
    elif hasattr(underlying_regressor, "get_params"):
        params_payload = underlying_regressor.get_params()
    else:
        params_payload = {"info": "Keras Sequential architecture configuration"}

    return {
        "model_metadata": {
            "model_uuid": model_uuid,
            "engine_type": engine_type,
            "description": description,
            "target_variable": target_col,
            "prediction_space_transformation": "10 ** prediction",
            "hyperparameters": params_payload,
        },
        "features": {
            "input_features": feature_names,
            "feature_importance": imp_dict
        },
        "validation_metrics": {
            "mean_cv_r2": float(np.mean(cv_metrics["r2"])),
            "std_cv_r2": float(np.std(cv_metrics["r2"])),
```

```
            "mean_cv_mae_km": float(np.mean(cv_metrics["mae_km"])),
        },
        "hypothesis_testing": {
            "delta_threshold_used": delta_threshold,
            "mean_delta_cv_r2": mean_delta_cv_r2,
            "statistical_outcome": statistical_outcome,
            "operational_outcome": operational_outcome
        }
    }

def _log_experiment_run(run_dict, engine_name, model_params, model_uuid, cfg,
test_mode: bool = False):
    import json

    is_test_mode = cfg.get("test_mode", False) or test_mode
    main_log_path = cfg["paths"]["experiment_log"].replace(".csv", "_test.csv") if
is_test_mode else cfg["paths"]["experiment_log"]
    folds_log_path = cfg["paths"]["experiment_folds_log"].replace(".csv", "_test.csv") if
is_test_mode else cfg["paths"]["experiment_folds_log"]

    timestamp_str = pd.Timestamp.now().strftime("%Y-%m-%d %H:%M:%S")
    params_string_lump = str(model_params)

    # Convert features to strict stringified JSON lists to preserve arrays inside single CSV
cells safely
    features_full_array = json.dumps(cfg["features"]["x_full"])
    features_restricted_array = json.dumps(cfg["features"]["x_restricted"])

    # 1. Expand the main aggregate model ledger row
    main_entry = {
        "timestamp": timestamp_str,
        "model_uuid": model_uuid,
        "engine_name": engine_name,

        # --- TARGET & FEATURE METADATA ARRAYS ---
        "target_col": cfg["features"]["target_col"],
        "target_raw_name": cfg["features"].get("target", "diameter"),
        "features_full_subset": features_full_array,
        "features_restricted_subset": features_restricted_array,

        # --- RAW SOURCE DATA METRICS ---
        "source_total_rows": run_dict.get("dataset_metrics", {}).get("total_rows"),
        "source_feature_count": run_dict.get("dataset_metrics", {}).get("feature_count"),
        "source_min_diameter_km": run_dict.get("dataset_metrics",
{}).get("min_diameter_km"),
        "source_max_diameter_km": run_dict.get("dataset_metrics",
{}).get("max_diameter_km"),
```

```
        # --- CROSS-VALIDATION AVERAGES ---
        "full_mean_cv_r2": run_dict["summary_averages"]["full_mean_cv_r2"],
        "restricted_mean_cv_r2": run_dict["summary_averages"]["restricted_mean_cv_r2"],
        "delta_threshold": run_dict["hypothesis_results"]["delta_threshold_used"],
        "calculated_delta_r2": run_dict["hypothesis_results"]["calculated_delta_r2"],
        "full_mean_cv_mae_km": run_dict["summary_averages"]
["full_mean_cv_mae_km"],
        "restricted_mean_cv_mae_km": run_dict["summary_averages"]
["restricted_mean_cv_mae_km"],

        # --- RECONCILIATION CONCLUSIONS ---
        "statistical_conclusion": run_dict["hypothesis_results"]["statistical"]["conclusion"],
        "operational_conclusion": run_dict["hypothesis_results"]["operational"]
["conclusion"],
        "model_parameters": params_string_lump
    }

    df_main = pd.DataFrame([main_entry])
    main_header = not os.path.exists(main_log_path)
    df_main.to_csv(main_log_path, mode="a", index=False, header=main_header)

    raw_folds = run_dict.get("raw_folds", {})
    fold_ids = raw_folds.get("fold_ids", [])

    if fold_ids:
        fold_records = []
        full_r2, rest_r2 = raw_folds.get("full_r2_per_fold", []),
raw_folds.get("restricted_r2_per_fold", [])
        delta_r2, full_mae, rest_mae = raw_folds.get("delta_r2_per_fold", []),
raw_folds.get("full_mae_km_per_fold", []), raw_folds.get("restricted_mae_km_per_fold",
[])

        for i in range(len(fold_ids)):
            fold_records.append({
                "timestamp": timestamp_str,
                "model_uuid": model_uuid,
                "engine_name": engine_name.upper(),
                "fold_id": fold_ids[i],

                # --- MATCHING SCHEMAS & SOURCE DATA ARRAYS PER SPLIT ---
                "target_col": cfg["features"]["target_col"],
                "target_raw_name": cfg["features"].get("target", "diameter"),
                "features_full_subset": features_full_array,
                "features_restricted_subset": features_restricted_array,
                "source_total_rows": run_dict.get("dataset_metrics", {}).get("total_rows"),
                "delta_threshold": run_dict["hypothesis_results"]["delta_threshold_used"],
```

```
            # --- OUT-OF-SAMPLE CROSS-VALIDATION SPLIT METRICS ---
            "full_model_r2": full_r2[i] if i < len(full_r2) else None,
            "restricted_model_r2": rest_r2[i] if i < len(rest_r2) else None,
            "delta_r2": delta_r2[i] if i < len(delta_r2) else None,
            "full_model_mae_km": full_mae[i] if i < len(full_mae) else None,
            "restricted_model_mae_km": rest_mae[i] if i < len(rest_mae) else None,
            "model_parameters": params_string_lump  # Placed explicitly as the final
terminal column field
        })

    df_folds = pd.DataFrame(fold_records)
    folds_header = not os.path.exists(folds_log_path)
    df_folds.to_csv(folds_log_path, mode="a", index=False, header=folds_header)

  print(f"Success! Main metrics logged to: {main_log_path}")
  print(f"Success! Cross-validation arrays logged to: {folds_log_path}")

def train_model(
  engine_type: str = "rf",
  dataset_cleaned_path: str = None,
  config_path: str = "config.yaml",
  override_params: Dict[str, Any] = None,
  override_x_full: List[str] = None,
  override_x_restricted: List[str] = None,
  delta_threshold: float = 0.05,
  test_mode: bool = False
) -> dict:

  engine_type_clean = engine_type.lower()
  if engine_type_clean.startswith("keras_") or engine_type_clean in ["wide_deep",
"resnet", "gated_resnet", "ft_transformer"]:
    if "gated_resnet" in engine_type_clean:
      architecture_type = "gated_resnet"
    elif "resnet" in engine_type_clean:
      architecture_type = "resnet"
    elif "ft_transformer" in engine_type_clean or engine_type_clean ==
"keras_ft_transformer":
      architecture_type = "ft_transformer"
    else:
      architecture_type = "wide_deep"

    print(f"[ROUTER] Passing pipeline execution to Keras Engine
({architecture_type.upper()})...")
    return train_keras_model(
      dataset_cleaned_path=dataset_cleaned_path,
      config_path=config_path,
```

```
            override_params=override_params,
            override_x_full=override_x_full,
            override_x_restricted=override_x_restricted,
            delta_threshold=delta_threshold,
            architecture_type=architecture_type,
            test_mode=test_mode
        )
    elif engine_type_clean in ["rf", "random_forest", "xgboost", "xgb", "gradient"]:
        print(f"[ROUTER] Passing pipeline execution to Traditional Tree Engine
({engine_type_clean.upper()})...")
        return train_traditional_model(
            engine_type=engine_type_clean,
            dataset_cleaned_path=dataset_cleaned_path,
            config_path=config_path,
            override_params=override_params,
            override_x_full=override_x_full,
            override_x_restricted=override_x_restricted,
            delta_threshold=delta_threshold,
            test_mode=test_mode
        )

    else:
        raise ValueError(
            f"Unsupported engine_type: '{engine_type}'. "
            f"Allowed values are: ['rf', 'xgboost', 'keras_wide_deep', 'keras_resnet',
'ft_transformer']"
        )

import os
import uuid
import hashlib
import joblib
import numpy as np
import pandas as pd
from typing import Union, List
import keras

def _log_predictions_to_csv(df_input: pd.DataFrame, predictions: np.ndarray,
engine_name: str, model_type: str, cfg: dict, is_test: bool, actual_uuid: str = None) ->
None:
    df_input = df_input.reset_index(drop=True)
    required_cols = ["spkid", "Asteroid", "Actual Diameter (km)"]
    for col in required_cols:
        if col not in df_input.columns:
            df_input[col] = np.nan if col != "Asteroid" else "Unknown"

    feature_key = "x_full" if model_type == "full" else "x_restricted"
```

```
    features_used = cfg.get("features", {}).get(feature_key, [])
    dataset_version = hashlib.md5("".join(sorted(features_used)).encode()).hexdigest()
[:8]
    model_uuid = actual_uuid if actual_uuid else f"{engine_name}_{model_type}
_{uuid.uuid4().hex[:8]}"

    actual_dia = df_input["Actual Diameter (km)"]
    error_pct = np.where(actual_dia > 0, (np.abs(actual_dia - predictions) / actual_dia) *
100, np.nan)

    df_log = pd.DataFrame({
        "spkid": df_input["spkid"],
        "Asteroid": df_input["Asteroid"],
        "Engine": engine_name,
        "Model_UUID": model_uuid,
        "Dataset_Version": dataset_version,
        "Subset": model_type,
        "Features_Used": "|".join(features_used),
        "Actual Diameter (km)": actual_dia,
        "Pred (km)": predictions,
        "Error %": error_pct
    })

    os.makedirs("logs", exist_ok=True)
    suffix = "_test.csv" if is_test else ".csv"
    full_path = os.path.join("logs", f"prediction_results{suffix}")

    file_exists = os.path.isfile(full_path)
    df_log.to_csv(full_path, mode="a", header=not file_exists, index=False)

def predict_model(input_data: Union[dict, pd.DataFrame], engine_type: str = "rf",
model_type: str = "full", config_path: str = "config.yaml", test_mode: bool = False) ->
Union[float, List[float], dict]:
    cfg = load_config(config_path)
    is_testing_mode = cfg.get("test_mode", False) or test_mode
    engine_type, model_type_clean = engine_type.lower(), model_type.lower()

    allowed_engines = ["rf", "xgboost", "xgb", "gradient", "keras_wide_deep",
"keras_resnet", "ft_transformer", "ft_t", "transformer", "keras_ft_transformer",
"keras_gated_resnet"]
    if engine_type not in allowed_engines:
        raise ValueError(f"engine_type must be one of: {allowed_engines}")

    if model_type_clean == "both":
        return {
            "full_model_prediction": predict_model(input_data, engine_type, "full",
config_path, test_mode=is_testing_mode),
```

```
            "restricted_model_prediction": predict_model(input_data, engine_type,
"restricted", config_path, test_mode=is_testing_mode)
        }

    if model_type_clean not in ["full", "restricted"]:
        raise ValueError("model_type must be either 'full', 'restricted', or 'both'.")

    df_input = pd.DataFrame([input_data]) if isinstance(input_data, dict) else
input_data.copy()

    base_model_dir = cfg["paths"]["model_dir"]
    run_type_dir = "testing" if is_testing_mode else "production"
    model_dir = os.path.join(base_model_dir, run_type_dir)

    if engine_type in ["ft_transformer", "ft_t", "transformer"]:
        engine_type = "keras_ft_transformer"

    extracted_uuid = None
    if engine_type.startswith("keras_"):
        model_path = os.path.join(model_dir, f"{engine_type}_{model_type_clean}
_model.keras")
        if not os.path.exists(model_path):
            raise FileNotFoundError(f"Production Keras binary missing at '{model_path}'.")
        model = keras.models.load_model(model_path)
        feature_key = "x_full" if model_type_clean == "full" else "x_restricted"
        X_matrix = df_input[cfg["features"][feature_key]].astype(np.float32).values
        log_prediction = model.predict(X_matrix, verbose=0).ravel()
        try: extracted_uuid = hashlib.md5(str(model.to_json()).encode()).hexdigest()[:12]
        except: pass
    else:
        model_path = os.path.join(model_dir, f"{engine_type}_{model_type_clean}
_model.pkl")
        if not os.path.exists(model_path) and f"{model_type_clean}_model" in cfg["paths"]:
            model_path = cfg["paths"][f"{model_type_clean}_model"]
        if not os.path.exists(model_path):
            raise FileNotFoundError(f"Production model binary missing at '{model_path}'.")
        artifacts = joblib.load(model_path)
        pipeline, metadata = artifacts["model"], artifacts["metadata"]
        extracted_uuid = metadata.get("model_metadata", {}).get("model_uuid", None)
        if not extracted_uuid:
            extracted_uuid = metadata.get("model_uuid", None)
        active_binary_engine = metadata["model_metadata"].get("engine_type",
engine_type).lower()
        requested_norm = "xgboost" if engine_type in ["xgboost", "xgb", "gradient"] else "rf"
        binary_norm = "xgboost" if active_binary_engine in ["xgboost", "xgb", "gradient"]
else "rf"
        if requested_norm != binary_norm:
```

```
            raise ValueError(f"Engine Mismatch! Disk is {binary_norm.upper()}, requested
{requested_norm.upper()}.")
        log_prediction = pipeline.predict(df_input)

    diameter_km = 10 ** log_prediction
    _log_predictions_to_csv(df_input, diameter_km, engine_type, model_type_clean, cfg,
is_testing_mode, actual_uuid=extracted_uuid)

    if isinstance(input_data, dict):
        return diameter_km.item() if isinstance(diameter_km, np.ndarray) else
float(diameter_km)
    return diameter_km.tolist()

#
================================================================
=====================
# SCIKIT-LEARN TRAINING ENGINE
#
================================================================
=====================
def train_traditional_model(
    engine_type: str = "rf",
    dataset_cleaned_path: str = None,
    config_path: str = "config.yaml",
    override_params: Dict[str, Any] = None,
    override_x_full: List[str] = None,
    override_x_restricted: List[str] = None,
    delta_threshold: float = 0.05,
    test_mode: bool = False
) -> dict:

    cfg = load_config(config_path)
    is_testing_mode = cfg.get("test_mode", False) or test_mode
    eval_cfg = cfg.get("evaluation_params", {})
    cv_splits = eval_cfg.get("cv_folds", 5)
    seed = eval_cfg.get("global_seed", 42)

    final_data_path = dataset_cleaned_path if dataset_cleaned_path is not None else
cfg["paths"]["dataset_cleaned"]
    if not os.path.exists(final_data_path):
        raise FileNotFoundError(f"Cleaned asteroid dataset not found at:
'{final_data_path}'")

    engine_type = engine_type.lower()
    if engine_type == "rf":
        ModelRegressor = RandomForestRegressor
        param_key = "rf_params"
```

```
        model_name_label = "RANDOM FOREST"
    elif engine_type in ["xgboost", "xgb", "gradient"]:
        ModelRegressor = GradientBoostingRegressor
        param_key = "xgb_params"
        model_name_label = "XGBOOST"
    else:
        raise ValueError("engine_type must be either 'rf' or 'xgboost'.")

    model_params = cfg["model_params"][param_key].copy()
    if override_params is not None:
        model_params.update(override_params)

    base_model_dir = cfg["paths"]["model_dir"]
    run_type_dir = "testing" if is_testing_mode else "production"
    run_dir = os.path.join(base_model_dir, run_type_dir)
    diagnostics_dir = os.path.join(run_dir, "diagnostics")
    os.makedirs(diagnostics_dir, exist_ok=True)

    # clean up old .txt and .json files
    old_txt_traces = glob.glob(os.path.join(diagnostics_dir, f"{engine_type}
_tree_rules_*.txt"))
    old_json_traces = glob.glob(os.path.join(diagnostics_dir, f"{engine_type}
_tree_rules_*.json"))
    for file_path in (old_txt_traces + old_json_traces):
        try:
            os.remove(file_path)
        except OSError:
            pass

    x_full = override_x_full if override_x_full is not None else cfg["features"]["x_full"]
    x_restricted = override_x_restricted if override_x_restricted is not None else
cfg["features"]["x_restricted"]
    target_col = cfg["features"]["target_col"]

    df_model = pd.read_csv(final_data_path)
    all_needed_features = list(set(x_full + x_restricted))
    X = df_model[all_needed_features]
    y = df_model[target_col].values.ravel()

    # Setup Validation
    print("\n" + "="*50)
    print(f"INITIALIZING {model_name_label} PIPELINE")
    print("="*50)
    print(f"Dataset Path:     {final_data_path}")
    print(f"Dataset Shape:    {df_model.shape}")
    print(f"Full Features:    {x_full}")
    print(f"Restrictive Features:  {x_restricted}")
```

```
    print(f"Target Column:    {target_col}")
    print(f"Model Parameters: {model_params}")
    print("="*50 + "\n")

    kf = KFold(n_splits=cv_splits, shuffle=True, random_state=seed)
    metrics = {"full": {"r2": [], "mae_km": []},"restricted": {"r2": [], "mae_km": []},"delta":
{"r2": []}}

    print(f"Running {model_name_label} Cross-Validation Evaluation...")
    for train_idx, test_idx in kf.split(X, y):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        rf_f = ModelRegressor(**model_params)
        rf_r = ModelRegressor(**model_params)

        rf_f.fit(X_train[x_full], y_train)
        preds_f = rf_f.predict(X_test[x_full])

        rf_r.fit(X_train[x_restricted], y_train)
        preds_r = rf_r.predict(X_test[x_restricted])

        f_r2 = r2_score(y_test, preds_f)
        r_r2 = r2_score(y_test, preds_r)

        metrics["full"]["r2"].append(f_r2)
        metrics["restricted"]["r2"].append(r_r2)
        metrics["delta"]["r2"].append(f_r2 - r_r2)

        y_test_km = 10 ** y_test
        metrics["full"]["mae_km"].append(mean_absolute_error(y_test_km, 10 ** preds_f))
        metrics["restricted"]["mae_km"].append(mean_absolute_error(y_test_km, 10 **
preds_r))

        current_fold = len(metrics["full"]["r2"])
        print(f"Fold {current_fold}/5 Complete | "
              f"Full R²: {f_r2:.4f} | "
              f"Restricted R²: {r_r2:.4f} | "
              f"Δ R²: {f_r2 - r_r2:.4f}")

    full_composer = ColumnTransformer(transformers=[("keep", "passthrough", x_full)],
remainder="drop")
    restricted_composer = ColumnTransformer(transformers=[("keep", "passthrough",
x_restricted)], remainder="drop")

    production_pipeline_full = Pipeline([("selector", full_composer),("regressor",
ModelRegressor(**model_params))])
```

```
    production_pipeline_restricted = Pipeline([("selector", restricted_composer),
("regressor", ModelRegressor(**model_params))])

    print(f"Running {model_name_label} production pipeline on full feature subset...")
    production_pipeline_full.fit(X, y)

    print(f"Running {model_name_label} production pipeline on restricted feature
subset...")
    production_pipeline_restricted.fit(X, y)

    mean_full_r2 = float(np.mean(metrics["full"]["r2"]))
    mean_restricted_r2 = float(np.mean(metrics["restricted"]["r2"]))
    mean_delta_cv_r2 = mean_full_r2 - mean_restricted_r2
    mean_full_mae = float(np.mean(metrics["full"]["mae_km"]))
    mean_restricted_mae = float(np.mean(metrics["restricted"]["mae_km"]))

    # Cross-Validation Averages Summary
    print("\n" + "="*50)
    print(f"CROSS-VALIDATION SUMMARY AVERAGES ({model_name_label})")
    print("="*50)
    print(f"Full Model Mean R²:       {mean_full_r2:.4f}")
    print(f"Restricted Model Mean R²: {mean_restricted_r2:.4f}")
    print(f"Mean Δ R² (Drop Impact):  {mean_delta_cv_r2:.4f} (± {np.std(metrics['delta']
['r2']):.4f})")
    print(f"Full Model Mean MAE:      {mean_full_mae:.4f} km")
    print(f"Restricted Model Mean MAE:{mean_restricted_mae:.4f} km")
    print("="*50 + "\n")

    if mean_delta_cv_r2 > 0:
        statistical_result = "Reject H0_stat: Group 2 features significantly improve out-of-
sample R²."
        statistical_outcome = "H1_stat"
    else:
        statistical_result = "Fail to Reject H0_stat: Group 2 features do not improve out-of-
sample R²."
        statistical_outcome = "H0_stat"

    if mean_delta_cv_r2 <= delta_threshold:
        operational_result = f"Reject H0_ops: Group 2 feature improvement is ≤ delta
({delta_threshold}). Restricted subset is operationally sufficient."
        operational_outcome = "H1_ops"
    else:
        operational_result = f"Fail to Reject H0_ops: Group 2 feature improvement is >
delta ({delta_threshold}). Restricted subset is insufficient."
        operational_outcome = "H0_ops"

    full_model_path = os.path.join(run_dir, f"{engine_type}_full_model.pkl")
```

```
    restricted_model_path = os.path.join(run_dir, f"{engine_type}_restricted_model.pkl")

    generated_uuid = str(uuid.uuid4())
    meta_args = (generated_uuid, engine_type, target_col, mean_delta_cv_r2,
delta_threshold, statistical_outcome, operational_outcome)

    full_payload = _generate_metadata_payload(production_pipeline_full, x_full, "Full
model with H.", metrics["full"], *meta_args)
    restricted_payload = _generate_metadata_payload(production_pipeline_restricted,
x_restricted, "Restricted model without H.", metrics["restricted"], *meta_args)

    full_model_path = os.path.join(run_dir, f"{engine_type}_full_model.pkl")
    restricted_model_path = os.path.join(run_dir, f"{engine_type}_restricted_model.pkl")

    joblib.dump({"model": production_pipeline_full, "metadata": full_payload},
full_model_path)
    joblib.dump({"model": production_pipeline_restricted, "metadata": restricted_payload},
restricted_model_path)
    print(f"Success! {model_name_label} main bundles saved safely.")

    models_to_unpack = [
        ("full", production_pipeline_full, x_full),
        ("restricted", production_pipeline_restricted, x_restricted)
    ]

    for subset_label, active_pipeline, active_features in models_to_unpack:
        underlying_regressor = active_pipeline.named_steps["regressor"]

        if hasattr(underlying_regressor, "estimators_"):
            raw_estimators = np.asarray(underlying_regressor.estimators_).ravel()

            def serialize_tree_to_dict(dt_estimator, node_index=0):
                tree_matrix = dt_estimator.tree_

                # Base Case: If the left child matches the right, it is a terminal leaf node
                if tree_matrix.children_left[node_index] ==
tree_matrix.children_right[node_index]:
                    return {
                        "node_type": "leaf",
                        "predicted_log_diameter_value": float(tree_matrix.value[node_index][0]
[0])
                    }

                # Recursive Step: Track the binary conditional branch splits using the active
features list
                return {
                    "node_type": "split",
```

```
                "split_feature": str(active_features[tree_matrix.feature[node_index]]),
                "split_threshold": float(tree_matrix.threshold[node_index]),
                "impurity_mse": float(tree_matrix.impurity[node_index]),
                "left_branch_if_less_equal": serialize_tree_to_dict(dt_estimator,
int(tree_matrix.children_left[node_index])),
                "right_branch_if_greater": serialize_tree_to_dict(dt_estimator,
int(tree_matrix.children_right[node_index]))
            }

        def serialize_tree_to_text(dt_estimator, node_index=0, depth=0):
            tree_matrix = dt_estimator.tree_
            indent = "|   " * depth

            # Base Case: It's a terminal leaf node
            if tree_matrix.children_left[node_index] ==
tree_matrix.children_right[node_index]:
                val = float(tree_matrix.value[node_index][0][0])
                return f"{indent}|--- value: [{val:.4f}]\n"

            # Extract branch metrics using active feature boundaries
            feat = active_features[tree_matrix.feature[node_index]]
            thresh = float(tree_matrix.threshold[node_index])

            left_child = int(tree_matrix.children_left[node_index])
            right_child = int(tree_matrix.children_right[node_index])

            # Recursively crawl left (<=) and right (>) paths
            rules_str = f"{indent}|--- {feat} <= {thresh:.4f}\n"
            rules_str += serialize_tree_to_text(dt_estimator, left_child, depth + 1)

            rules_str += f"{indent}|--- {feat} >  {thresh:.4f}\n"
            rules_str += serialize_tree_to_text(dt_estimator, right_child, depth + 1)

            return rules_str

        print(f"Exporting raw rule maps for ALL {len(raw_estimators)} individual trees
into {subset_label.upper()} spaces...")
        for idx, tree_estimator in enumerate(raw_estimators):
            # --- 1. EXPORT PURE PYTHON TEXT LAYOUT ---
            tree_text_layout = serialize_tree_to_text(tree_estimator)
            tree_txt_name = f"{engine_type}_{subset_label}_tree_rules_{idx:03d}.txt"
            with open(os.path.join(diagnostics_dir, tree_txt_name), "w") as f_tree:
                f_tree.write(f"{model_name_label} ({subset_label.upper()}) Tree Index {idx}
Decisions:\n" + "="*50 + "\n")
                f_tree.write(tree_text_layout)

            # --- 2. EXPORT PURE PYTHON JSON LAYOUT ---
```

```
            tree_dictionary = serialize_tree_to_dict(tree_estimator)
            tree_json_name = f"{engine_type}_{subset_label}_tree_rules_{idx:03d}.json"
            with open(os.path.join(diagnostics_dir, tree_json_name), "w") as f_json:
                json.dump(tree_dictionary, f_json, indent=4)
    print(f"Success! Flat rule structures (.txt and .json) for both Full and Restricted
models updated safely inside: {diagnostics_dir}")

    print("\n" + "="*23 + f" HYPOTHESIS TESTING REPORT ({model_name_label}) " +
"="*22)
    print(f"Testing against Operational Tolerance (δ) = {delta_threshold}")
    print(f"Calculated Delta R² (Drop Impact):       = {mean_delta_cv_r2:.4f}")
    print("-" * 72)
    print(f"Statistical Test: [{statistical_outcome}] -> {statistical_result}")
    print(f"Operational Test: [{operational_outcome}] -> {operational_result}")
    print("=" * 72 + "\n")

    output_payload = {
        "status": "success",
        "model_uuid": generated_uuid,

        "dataset_metrics": {
            "total_rows": int(df_model.shape[0] if hasattr(df_model, 'shape') else
len(df_model)),
            "feature_count": int(len(x_full)),
            "min_diameter_km": float(10 ** np.min(y)),
            "max_diameter_km": float(10 ** np.max(y))
        },

        "model_paths": {"full_model": full_model_path, "restricted_model":
restricted_model_path},
        "hypothesis_results": {
            "delta_threshold_used": delta_threshold,
            "calculated_delta_r2": mean_delta_cv_r2,
            "statistical": {"outcome": statistical_outcome, "conclusion": statistical_result},
            "operational": {"outcome": operational_outcome, "conclusion":
operational_result}
        },
        "summary_averages": {
            "full_mean_cv_r2": mean_full_r2,
            "restricted_mean_cv_r2": mean_restricted_r2,
            "delta_std_cv_r2": float(np.std(metrics["delta"]["r2"])),
            "full_mean_cv_mae_km": mean_full_mae,
            "restricted_mean_cv_mae_km": mean_restricted_mae
        },
        "raw_folds": {
            "fold_ids": list(range(1, 6)),
            "full_r2_per_fold": [float(v) for v in metrics["full"]["r2"]],
```

```
            "restricted_r2_per_fold": [float(v) for v in metrics["restricted"]["r2"]],
            "delta_r2_per_fold": [float(v) for v in metrics["delta"]["r2"]],
            "full_mae_km_per_fold": [float(v) for v in metrics["full"]["mae_km"]],
            "restricted_mae_km_per_fold": [float(v) for v in metrics["restricted"]["mae_km"]]
        }
    }

    _log_experiment_run(run_dict=output_payload, engine_name=engine_type,
model_params=model_params, model_uuid=generated_uuid,
cfg=cfg,test_mode=is_testing_mode)
    return output_payload

#
================================================================
======================
# KERAS TRAINING ENGINE
#
================================================================
======================
def build_wide_and_deep(
    input_dim: int,
    config: dict,
    norm_layer: layers.Normalization = None,
    **kwargs
) -> keras.Model:

    keras_cfg = config["model_params"]["keras_nn"]
    wd_cfg = keras_cfg["wide_and_deep"]

    act_func = kwargs.get("activation", keras_cfg["activation"])
    lr = kwargs.get("learning_rate", keras_cfg["learning_rate"])

    w_dim = kwargs.get("wide_dim", wd_cfg["wide_dim"])
    d_dims = kwargs.get("deep_dims", wd_cfg["deep_dims"])
    drop_rate = kwargs.get("dropout_rate", wd_cfg["dropout_rate"])

    inputs = layers.Input(shape=(input_dim,))
    x = norm_layer(inputs) if norm_layer is not None else inputs

    wide = layers.Dense(w_dim, activation="linear")(x)
    deep = x
    for dim in d_dims:
        deep = layers.Dense(dim, activation=act_func)(deep)
        deep = layers.BatchNormalization()(deep)
        deep = layers.Dropout(drop_rate)(deep)

    merged = layers.concatenate([wide, deep])
```

```
    outputs = layers.Dense(1)(merged)

    model = keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=lr),
        loss=keras_cfg["loss"],
        metrics=keras_cfg["metrics"],
        run_eagerly=True
    )
    return model


def build_tabular_resnet(
    input_dim: int,
    config: dict,
    norm_layer: layers.Normalization = None,
    **kwargs
) -> keras.Model:

    keras_cfg = config["model_params"]["keras_nn"]
    res_cfg = keras_cfg["tabular_resnet"]

    act_func = kwargs.get("activation", keras_cfg["activation"])
    lr = kwargs.get("learning_rate", keras_cfg["learning_rate"])

    h_dim = kwargs.get("hidden_dim", res_cfg["hidden_dim"])
    blocks = kwargs.get("num_blocks", res_cfg["num_blocks"])
    drop_rate = kwargs.get("dropout_rate", res_cfg["dropout_rate"])

    inputs = layers.Input(shape=(input_dim,))
    x = norm_layer(inputs) if norm_layer is not None else inputs
    x = layers.Dense(h_dim, activation=act_func)(x)
    x = layers.BatchNormalization()(x)

    for _ in range(blocks):
        res = layers.Dense(h_dim, activation=act_func)(x)
        res = layers.BatchNormalization()(res)
        res = layers.Dropout(drop_rate)(res)
        x = layers.add([x, res])
    outputs = layers.Dense(1)(x)

    model = keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=lr),
        loss=keras_cfg["loss"],
        metrics=keras_cfg["metrics"],
        run_eagerly=True
```

```
    )
    return model

def build_keras_ft_transformer(input_dim, config, norm_layer=None,
learning_rate=0.001, **kwargs):

    ft_cfg = config.get("model_params", {}).get("keras_nn", {}).get("ft_transformer", {})
    num_heads = kwargs.get("num_heads", ft_cfg.get("num_heads", 4))
    num_attn_blocks = kwargs.get("num_attn_blocks", ft_cfg.get("num_attn_blocks", 3))
    embedding_dim = kwargs.get("embedding_dim", ft_cfg.get("embedding_dim", 32))
    dropout_rate = kwargs.get("dropout_rate", ft_cfg.get("dropout_rate", 0.1))

    inputs = layers.Input(shape=(input_dim,))
    x = norm_layer(inputs) if norm_layer is not None else inputs

    tokenized_features = []
    for i in range(input_dim):
        feat_slice = x[:, i:i+1]
        token_emb = layers.Dense(embedding_dim)(feat_slice)
        token_emb = layers.Reshape((1, embedding_dim))(token_emb)
        tokenized_features.append(token_emb)

    tokens = layers.Concatenate(axis=1)(tokenized_features)

    for _ in range(num_attn_blocks):
        attn_output = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embedding_dim)(tokens, tokens)
        attn_output = layers.Dropout(dropout_rate)(attn_output)
        tokens = layers.Add()([tokens, attn_output])
        tokens = layers.LayerNormalization()(tokens)

        ffn_output = layers.Dense(embedding_dim * 2, activation="relu")(tokens)
        ffn_output = layers.Dense(embedding_dim)(ffn_output)
        ffn_output = layers.Dropout(dropout_rate)(ffn_output)
        tokens = layers.Add()([tokens, ffn_output])
        tokens = layers.LayerNormalization()(tokens)

    flattened = layers.Flatten()(tokens)
    dense_out = layers.Dense(128, activation="relu")(flattened)
    dense_out = layers.Dropout(0.2)(dense_out)
    outputs = layers.Dense(1, activation="linear")(dense_out)

    model = keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=learning_rate),
        loss="mse",
        metrics=["mae"]
```

```
    )
    return model

def build_keras_gated_resnet(input_dim, config, norm_layer=None,
learning_rate=0.001, **kwargs):
    res_cfg = config.get("model_params", {}).get("keras_nn", {}).get("tabular_resnet", {})
    hidden_dim = kwargs.get("hidden_dim", res_cfg.get("hidden_dim", 256))
    num_blocks = kwargs.get("num_blocks", res_cfg.get("num_blocks", 3))
    dropout_rate = kwargs.get("dropout_rate", res_cfg.get("dropout_rate", 0.2))

    inputs = layers.Input(shape=(input_dim,))
    x = norm_layer(inputs) if norm_layer is not None else inputs

    feature_gate = layers.Dense(input_dim, activation="sigmoid")(x)
    feature_lane = layers.Dense(input_dim, activation="linear")(x)

    gated_features = layers.Multiply()([feature_lane, feature_gate])

    res_input = layers.Dense(hidden_dim, activation="relu")(gated_features)
    x_res = res_input

    for _ in range(num_blocks):
        res = layers.Dense(hidden_dim, activation="relu")(x_res)
        res = layers.Dropout(dropout_rate)(res)
        res = layers.Dense(hidden_dim)(res)

        x_res = layers.Add()([x_res, res])
        x_res = layers.BatchNormalization()(x_res)

    outputs = layers.Dense(1, activation="linear")(x_res)

    model = keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=keras.optimizers.AdamW(learning_rate=learning_rate),
        loss="mse",
        metrics=["mae"]
    )
    return model

def train_keras_model(
    dataset_cleaned_path: str = None,
    config_path: str = "config.yaml",
    override_params: Dict[str, Any] = None,
    override_x_full: List[str] = None,
    override_x_restricted: List[str] = None,
    delta_threshold: float = 0.05,
    architecture_type: str = "wide_deep",
```

```
    test_mode: bool = False
) -> dict:

    cfg = load_config(config_path)
    is_testing_mode = cfg.get("test_mode", False) or test_mode
    eval_cfg = cfg.get("evaluation_params", {})
    cv_splits = eval_cfg.get("cv_folds", 5)
    seed = eval_cfg.get("global_seed", 42)


    final_data_path = dataset_cleaned_path if dataset_cleaned_path is not None else
cfg["paths"]["dataset_cleaned"]
    if not os.path.exists(final_data_path):
        raise FileNotFoundError(f"Cleaned asteroid dataset not found at:
'{final_data_path}'")

    engine_type = f"keras_{architecture_type}"
    model_name_label = f"KERAS {architecture_type.upper()} NETWORK"

    train_cfg = cfg.get("training_params", {})
    nn_cfg = cfg.get("model_params", {}).get("keras_nn", {})

    model_params = {
        "epochs": train_cfg.get("epochs", 100),
        "batch_size": train_cfg.get("batch_size", 64),
        "learning_rate": nn_cfg.get("learning_rate", 0.001),
        "patience": train_cfg.get("early_stopping", {}).get("patience", 10),
        "restore_best_weights": train_cfg.get("early_stopping",
{}).get("restore_best_weights", True)
    }

    if override_params is not None: model_params.update(override_params)
    reserved_keys = ["epochs", "batch_size", "learning_rate", "patience",
"restore_best_weights"]
    builder_overrides = {k: v for k, v in model_params.items() if k not in reserved_keys}

    base_model_dir = cfg["paths"]["model_dir"]
    run_type_dir = "testing" if is_testing_mode else "production"
    run_dir = os.path.join(base_model_dir, run_type_dir)
    os.makedirs(run_dir, exist_ok=True)

    x_full = override_x_full if override_x_full is not None else cfg["features"]["x_full"]
    x_restricted = override_x_restricted if override_x_restricted is not None else
cfg["features"]["x_restricted"]
    target_col = cfg["features"]["target_col"]

    df_raw = pd.read_csv(final_data_path)
```

```
    df_model = _engineer_tabular_interactions(df_raw)

    X_full_raw = df_model[x_full].astype(np.float32)
    X_rest_raw = df_model[x_restricted].astype(np.float32)
    y = df_model[target_col].values.ravel().astype(np.float32)

    X_full_numpy = X_full_raw.values
    X_rest_numpy = X_rest_raw.values

    print("\n" + "="*50)
    print(f"INITIALIZING {model_name_label} PIPELINE")
    print("="*50)
    print(f"Dataset Path:     {final_data_path}")
    print(f"Dataset Shape:    {df_model.shape}")
    print(f"Full Features:    {x_full}")
    print(f"Restrictive Features:  {x_restricted}")
    print(f"Target Column:    {target_col}")
    print(f"Model Parameters: {model_params}")
    print("="*50 + "\n")

    if architecture_type == "wide_deep":
        builder_func = build_wide_and_deep
    elif architecture_type == "resnet":
        builder_func = build_tabular_resnet
    elif architecture_type == "gated_resnet":
        builder_func = build_keras_gated_resnet  # <-- HOOKED UP EXPLICIT LAYER
POINTER
    elif architecture_type == "ft_transformer":
        builder_func = build_keras_ft_transformer
    else:
        raise ValueError(f"Unknown Keras architecture type: {architecture_type}")

    kf = KFold(n_splits=cv_splits, shuffle=True, random_state=seed)
    metrics = {"full": {"r2": [], "mae_km": []},"restricted": {"r2": [], "mae_km": []},"delta":
{"r2": []}}

    optimal_epochs_f = []
    optimal_epochs_r = []

    print(f"Running {model_name_label} Cross-Validation Evaluation...")
    for current_fold, (train_idx, test_idx) in enumerate(kf.split(df_model, y), 1):
        print(f"\n--- Starting Fold {current_fold}/5 Initialization ---")
        X_train_f, X_test_f = X_full_numpy[train_idx], X_full_numpy[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        norm_f = layers.Normalization(axis=-1)
        norm_f.adapt(X_train_f)
```

```
        model_f = builder_func(
            input_dim=len(x_full),
            config=cfg,
            norm_layer=norm_f,
            learning_rate=model_params["learning_rate"],
            **builder_overrides
        )

        early_stopping = keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=model_params["patience"],
            restore_best_weights=model_params["restore_best_weights"]
        )

        history_f = model_f.fit(X_train_f, y_train,
validation_split=cfg["data_params"].get("val_split", 0.1),
epochs=model_params["epochs"], batch_size=model_params["batch_size"],
callbacks=[early_stopping], verbose=1)

        stopped_epoch_f = len(history_f.history["loss"])
        best_epoch_f = max(1, stopped_epoch_f - model_params["patience"]) if
early_stopping.stopped_epoch > 0 else stopped_epoch_f
        optimal_epochs_f.append(best_epoch_f)

        preds_f = model_f.predict(X_test_f, verbose=0).ravel()
        if is_testing_mode:
            preds_f = np.clip(preds_f, -5.0, 6.0)

        X_train_r, X_test_r = X_rest_numpy[train_idx], X_rest_numpy[test_idx]

        norm_r = layers.Normalization(axis=-1)
        norm_r.adapt(X_train_r)

        model_r = builder_func(
            input_dim=len(x_restricted),
            config=cfg,
            norm_layer=norm_r,
            learning_rate=model_params["learning_rate"],
            **builder_overrides
        )

        history_r = model_r.fit(X_train_r, y_train,
validation_split=cfg["data_params"].get("val_split", 0.1),
epochs=model_params["epochs"], batch_size=model_params["batch_size"],
callbacks=[early_stopping], verbose=1)
```

```
        stopped_epoch_r = len(history_r.history["loss"])
        best_epoch_r = max(1, stopped_epoch_r - model_params["patience"]) if
early_stopping.stopped_epoch > 0 else stopped_epoch_r
        optimal_epochs_r.append(best_epoch_r)

        preds_r = model_r.predict(X_test_r, verbose=0).ravel()
        if is_testing_mode:
            preds_r = np.clip(preds_r, -5.0, 6.0)

        f_r2 = r2_score(y_test, preds_f)
        r_r2 = r2_score(y_test, preds_r)

        metrics["full"]["r2"].append(f_r2)
        metrics["restricted"]["r2"].append(r_r2)
        metrics["delta"]["r2"].append(f_r2 - r_r2)

        y_test_km = 10 ** y_test
        metrics["full"]["mae_km"].append(mean_absolute_error(y_test_km, 10 ** preds_f))
        metrics["restricted"]["mae_km"].append(mean_absolute_error(y_test_km, 10 **
preds_r))
        print(f"Fold {len(metrics['full']['r2'])}/5 Complete | Full R²: {f_r2:.4f} | Restricted R²:
{r_r2:.4f} | Δ R²: {f_r2 - r_r2:.4f}")

    prod_epochs_f = int(np.ceil(np.mean(optimal_epochs_f) * 1.10))
    prod_epochs_r = int(np.ceil(np.mean(optimal_epochs_r) * 1.10))

    prod_epochs_f = _calculate_production_epochs(optimal_epochs_f,
model_params["epochs"])
    prod_epochs_r = _calculate_production_epochs(optimal_epochs_r,
model_params["epochs"])

    print(f"  -> Production Hard Caps Locked At: {prod_epochs_f} epochs (Full) |
{prod_epochs_r} epochs (Rest)")

    final_norm_f = layers.Normalization(axis=-1)
    final_norm_f.adapt(X_full_raw.values)

    final_norm_r = layers.Normalization(axis=-1)
    final_norm_r.adapt(X_rest_raw.values)

    prod_model_full = builder_func(input_dim=len(x_full), config=cfg,
norm_layer=final_norm_f, learning_rate=model_params["learning_rate"],
**builder_overrides)
    prod_model_restricted = builder_func(input_dim=len(x_restricted), config=cfg,
norm_layer=final_norm_r, learning_rate=model_params["learning_rate"],
**builder_overrides)
```

```
    print(f"\nTraining final production {model_name_label} models on full feature
dataset...")
    prod_model_full.fit(X_full_raw.values, y, epochs=prod_epochs_f,
batch_size=model_params["batch_size"], verbose=1)

    print(f"\nTraining final production {model_name_label} models on restricted feature
dataset...")
    prod_model_restricted.fit(X_rest_raw.values, y, epochs=prod_epochs_r,
batch_size=model_params["batch_size"], verbose=1)

    mean_full_r2 = float(np.mean(metrics["full"]["r2"]))
    mean_restricted_r2 = float(np.mean(metrics["restricted"]["r2"]))
    mean_delta_cv_r2 = float(mean_full_r2 - mean_restricted_r2)
    mean_full_mae = float(np.mean(metrics["full"]["mae_km"]))
    mean_restricted_mae = float(np.mean(metrics["restricted"]["mae_km"]))

    print("\n" + "="*50)
    print(f"CROSS-VALIDATION SUMMARY AVERAGES ({architecture_type.upper()})")
    print("="*50)
    print(f"Full Model Mean R²:       {mean_full_r2:.4f}")
    print(f"Restricted Model Mean R²: {mean_restricted_r2:.4f}")
    print(f"Mean Δ R² (Drop Impact):  {mean_delta_cv_r2:.4f} (±
{float(np.std(metrics['delta']['r2'])):.4f})")
    print(f"Full Model Mean MAE:      {mean_full_mae:.4f} km")
    print(f"Restricted Model Mean MAE:{mean_restricted_mae:.4f} km")
    print("="*50 + "\n")

    statistical_outcome = "H1_stat" if mean_delta_cv_r2 > 0 else "H0_stat"
    statistical_result = "Reject H0_stat: Group 2 features significantly improve out-of-
sample R²." if mean_delta_cv_r2 > 0 else "Fail to Reject H0_stat: Group 2 features do
not improve out-of-sample R²."

    operational_outcome = "H1_ops" if mean_delta_cv_r2 <= delta_threshold else
"H0_ops"
    operational_result = f"Reject H0_ops: Group 2 feature improvement is ≤ delta
({delta_threshold}). Restricted subset is operationally sufficient." if mean_delta_cv_r2 <=
delta_threshold else f"Fail to Reject H0_ops: Group 2 feature improvement is > delta
({delta_threshold}). Restricted subset is insufficient."

    print("\n" + "="*23 + f" HYPOTHESIS TESTING REPORT
({architecture_type.upper()}) " + "="*22)
    print(f"Testing against Operational Tolerance (δ) = {delta_threshold:.4f}")
    print(f"Calculated Delta R² (Drop Impact):        = {mean_delta_cv_r2:.4f}")
    print("-" * 72)
    print(f"Statistical Test: [{statistical_outcome}] -> {statistical_result}")
    print(f"Operational Test: [{operational_outcome}] -> {operational_result}")
    print("=" * 72 + "\n")
```

```
    generated_uuid = str(uuid.uuid4())
    class MockPipeline:
        def __init__(self, regressor):
            self.named_steps = {"regressor": regressor}

    mock_pipe_f = MockPipeline(prod_model_full)
    mock_pipe_r = MockPipeline(prod_model_restricted)

    delta_threshold = float(delta_threshold)
    meta_args = (generated_uuid, engine_type, target_col, model_params,
mean_delta_cv_r2, delta_threshold, statistical_outcome, operational_outcome)

    full_payload = _generate_metadata_payload(mock_pipe_f, x_full, f"Full
{architecture_type} network.", metrics["full"], *meta_args)
    restricted_payload = _generate_metadata_payload(mock_pipe_r, x_restricted,
f"Restricted {architecture_type} network.", metrics["restricted"], *meta_args)

    full_model_path = os.path.join(run_dir, f"{engine_type}_full_model.keras")
    restricted_model_path = os.path.join(run_dir, f"{engine_type}
_restricted_model.keras")

    prod_model_full.save(full_model_path, metadata=full_payload)
    prod_model_restricted.save(restricted_model_path, metadata=restricted_payload)
    print(f"Success! Keras models production bundles saved safely to: {run_dir}")

    output_payload = {
        "status": "success",
        "model_uuid": generated_uuid,

        "dataset_metrics": {
            "total_rows": int(df_model.shape[0] if hasattr(df_model, 'shape') else
len(df_model)),
            "feature_count": int(len(x_full)),
            "min_diameter_km": float(10 ** np.min(y)),
            "max_diameter_km": float(10 ** np.max(y))
        },

        "model_paths": {
            "full_model": full_model_path,
            "restricted_model": restricted_model_path
        },
        "hypothesis_results": {
            "delta_threshold_used": delta_threshold,
            "calculated_delta_r2": mean_delta_cv_r2,
            "statistical": {"outcome": statistical_outcome, "conclusion": statistical_result},
            "operational": {"outcome": operational_outcome, "conclusion":
```

```
operational_result}
        },
        "summary_averages": {
            "full_mean_cv_r2": mean_full_r2,
            "restricted_mean_cv_r2": mean_restricted_r2,
            "delta_std_cv_r2": float(np.std(metrics["delta"]["r2"])),
            "full_mean_cv_mae_km": mean_full_mae,
            "restricted_mean_cv_mae_km": mean_restricted_mae
        },
        "raw_folds": {
            "fold_ids": list(range(1, 6)),
            "full_r2_per_fold": [float(v) for v in metrics["full"]["r2"]],
            "restricted_r2_per_fold": [float(v) for v in metrics["restricted"]["r2"]],
            "delta_r2_per_fold": [float(v) for v in metrics["delta"]["r2"]],
            "full_mae_km_per_fold": [float(v) for v in metrics["full"]["mae_km"]],
            "restricted_mae_km_per_fold": [float(v) for v in metrics["restricted"]["mae_km"]]
        }
    }

    _log_experiment_run(run_dict=output_payload, engine_name=engine_type,
model_params=model_params, model_uuid=generated_uuid, cfg=cfg,
test_mode=is_testing_mode)
    return output_payload
```